PROJECT REPORT

ON

“Sound for Silence”

Submitted to:

R. T.U., KOTA (RAJASTHAN)

In partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

(Insert College Logo Here)

Under the Supervision of:

Er. Pawan Sen HOD, CSE

Submitted by:

[Your Name] [RTU Roll No]

ARYA COLLEGE OF ENGINEERING, JAIPUR RAJASTHAN TECHNICAL UNIVERSITY, KOTA (RAJASTHAN) (2025-26)

ARYA COLLEGE OF ENGINEERING, JAIPUR

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Certificate of Completion

This is to certify that the project titled **"Sound for Silence"** has been successfully completed by **[[Your Name]]**, a student of **Bachelor of Technology, 8th Semester**, at **Arya College of Engineering, Kukas, Jaipur**.

The project work presented in this report is a **bonafide and satisfactory account** of the work carried out under my supervision, and is hereby **recommended for submission** as part of the fulfilment of the academic requirements for the 8th semester of the B.Tech program.

Project In-charge **[Your Name] with [Roll Number]**

Er Narender Narwal

DECLARATION BY THE CANDIDATE

I, **[[Your Name]]**, hereby declare that the project work titled **"Sound for Silence"** is an original and authentic piece of work carried out by me at **Arya College of Engineering, Jaipur** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)**.

I further affirm that this work has not been submitted, either fully or partially, for the award of any other degree or diploma at any institution.

Name: [[Your Name]]

Roll No.: [[RTU Roll No]]

ACKNOWLEDGEMENT

It gives me immense pleasure to express my heartfelt gratitude to all those who have supported and guided me throughout the successful completion of this final year project.

I would first like to extend my sincere thanks to **Dr. Arvind Agarwal**, President, and **Dr. Puja Agarwal**, Group Director of Arya College of Engineering, Kukas, Jaipur, for providing an excellent academic environment and all the necessary facilities that foster learning and innovation.

I am deeply grateful to our respected **Principal, Dr. Himanshu Arora**, for his continuous encouragement and administrative support. I would also like to express my special thanks to **Er. Pawan Sen**, Head of the Department, Computer Science and Engineering, for his visionary leadership, valuable suggestions, and motivation throughout the course of this project.

I am also thankful to my Project In-charge, **Er Narender Narwal**, whose timely feedback, guidance, and coordination played a vital role in ensuring the quality and completion of this project work.

Last but not least, I sincerely thank all the faculty members, lab assistants, and my peers who contributed directly or indirectly with their support and inspiration during the entire project duration.

This project has been a significant learning experience and a stepping stone in my professional development.

[STUDENT NAME] [ROLL NO] B.TECH 8th SEM

TABLE OF CONTENTS

1. Introduction 1
   1.1. Abstract of Project 1
      1.1.1. Title of the Project 1
      1.1.2. Objective 1
   1.2. Problem Specification/Need of Project 1
2. Feasibility Study 3
3. Software Requirement Specifications 4
   3.1. Introduction 4
   3.2. Selection of Technology/Specific Requirements 4
4. Design 5
   4.1. ER Diagram 5
   4.2. Data Flow Diagram (0 & 1 Level) 5
   4.3. Modules 6
   4.4. Database 6
   4.5. Input-Output form (Screen Layout) 7
5. Implementation/Technological Environment 8
6. Testing & Results 9
7. Limitations 10
8. Conclusion & Future Scope 11

LIST OF TABLES

LIST OF FIGURES

Figure 1 Conceptual Entity-Relationship Model 5
Figure 2 Data Flow Diagram Placeholder 6

1. Introduction

1.1. Abstract of Project

Sound for Silence is a dual-interface digital healthcare platform designed specifically for children undergoing cognitive, auditory, and speech therapy. The platform targets children who have received cochlear implants and need structured, monitored rehabilitation over an extended period. This report focuses specifically on the backend architecture, cloud infrastructure, and security paradigms developed to support this ecosystem.

As the Backend & Firebase Engineer, my primary contribution involved architecting a scalable, serverless backend using Firebase. This encompassed implementing secure authentication workflows, designing an efficient NoSQL database schema on Firestore, configuring role-based access control (RBAC) via Firebase Security Rules, and managing the deployment pipeline for the web application using Firebase Hosting. The unified backend ensures real-time data synchronization between the Android mobile application (patient portal) and the web-based dashboard (therapist portal), delivering a seamless experience for children, parents, and clinicians without the overhead of maintaining a dedicated monolithic server.

1.1.1. Title of the Project

Sound for Silence

1.1.2. Objective

- To implement a unified Firebase backend providing secure, role-based authentication.
- To design a scalable NoSQL database schema using Firestore for storing user profiles, clinical assessments, therapy progress, and video metadata.
- To enforce rigorous data privacy and access control using Firebase Security Rules, ensuring therapists only access their assigned patients and parents only view their child's data.
- To deploy and host the therapist web dashboard using Firebase Hosting with high availability.
- To ensure real-time data synchronization between the Android application and the web dashboard.

1.2. Problem Specification/Need of Project

Children with cochlear implants require consistent, guided practice over months and years. Currently, therapy progress is often tracked manually using paper records, which makes longitudinal monitoring difficult, error-prone, and disconnected from the daily exercises performed at home.

From a system architecture perspective, building a traditional monolithic backend to manage real-time progress tracking, secure multi-role access (therapists, parents, children), and dynamic video content delivery is resource-intensive and expensive to host. There is a critical need for a lightweight, serverless backend solution that can handle real-time data synchronization across disparate platforms (Android and Web) while maintaining strict healthcare data privacy standards. Sound for Silence addresses this gap by utilizing Firebase to provide a robust, real-time, and secure backend infrastructure.

2. Feasibility Study

Technical Feasibility: The backend system is technically feasible using Firebase, a comprehensive app development platform backed by Google. Firebase Authentication provides out-of-the-box support for email/password login and secure session management. Firestore, a NoSQL document database, natively supports real-time listeners, which is crucial for syncing data between the app and the dashboard. Firebase Security Rules allow for fine-grained, serverless access control, eliminating the need for complex middleware.

Operational Feasibility: The serverless nature of the Firebase backend means there is no need for ongoing server maintenance, operating system patching, or manual scaling. The automated syncing reduces the need for manual data entry by therapists, making the system highly operationally efficient. The infrastructure is designed to automatically scale with the number of concurrent users, making it practical for clinics and rehabilitation centers of any size.

Economic Feasibility: The project relies on Firebase's free tier (Spark Plan), which offers generous quotas for authentication, Firestore reads/writes (50,000 reads/20,000 writes per day), and hosting. This ensures that the development, testing, and initial deployment phases incur zero backend infrastructure costs, making it highly economically viable for a student project and scalable for real-world deployment.

3. Software Requirement Specifications

3.1. Introduction

This section outlines the specific requirements for the backend infrastructure of the Sound for Silence platform. It defines the core responsibilities of the Firebase architecture, including authentication flows, data storage, and access policies.

3.2. Selection of Technology/Specific Requirements

The backend is entirely serverless and relies on the following technologies:

- **Firebase Authentication:** For managing user identities securely. It handles user registration, login, and password management for both therapists and parents.
- **Firebase Cloud Firestore:** A flexible, scalable NoSQL cloud database used to store and sync data for client- and server-side development. Selected for its real-time capabilities and offline support.
- **Firebase Security Rules:** A custom rules engine used to dictate who has read and write access to specific documents in Firestore based on user authentication status and custom claims.
- **Firebase Hosting:** Fast and secure web hosting for the therapist dashboard, providing an SSL certificate out of the box and utilizing a global CDN.

4. Design

4.1. ER Diagram

While Firestore is a NoSQL database, the conceptual relationships between entities can be visualized similarly to an ER diagram.

(Insert ER Diagram showing relationships between Users, Patients, Stages, Videos, and Assessments)

Figure 1: Conceptual Entity-Relationship Model

4.2. Data Flow Diagram (0 & 1 Level)

Level 0 DFD (Context Diagram): The Level 0 DFD illustrates the interaction between the primary entities (Parent/Child via Android App, Therapist via Web Dashboard) and the central Firebase Backend System.

Level 1 DFD: The Level 1 DFD breaks down the Firebase backend into its core services: Authentication, Firestore, and Hosting, showing how data requests, authentication tokens, and real-time updates flow between the client applications and the backend services.

(Insert DFD Level 0 & 1 Here)

Figure 2: Data Flow Diagram Placeholder

4.3. Modules

The backend architecture is divided into the following key modules:

1. **Authentication Module:** Handles sign-up, login, and token generation. Differentiates user roles (e.g., assigning a 'therapist' or 'parent' role).
2. **Data Storage Module:** Manages the CRUD operations for user profiles, therapy stages, multimedia metadata, quiz results, and CAP/SIR clinical scores.
3. **Security & Access Control Module:** Evaluates every incoming request against Firebase Security Rules to ensure data privacy and prevent unauthorized access.
4. **Hosting & Deployment Module:** Manages the deployment of the Web Dashboard assets to Firebase's global edge network.

4.4. Database

The Firestore database uses a document-oriented NoSQL schema. The data is structured into root-level collections and sub-collections:

- **`users` Collection:** Stores user profiles, containing fields like `uid`, `email`, `role`, and `name`.
- **`patients` Collection:** Stores patient details. Linked to a parent's `uid` and assigned to a therapist's `uid`.
- **`assessments` Sub-collection (under `patients`):** Stores the chronological CAP and SIR scores entered by therapists.
- **`progress` Sub-collection (under `patients`):** Stores daily streaks, stage completion status, and quiz scores automatically updated by the Android app.
- **`stages` Collection:** Contains the curriculum metadata, including stage order, video URLs, and required prerequisites.

4.5. Input-Output form (Screen Layout)

As a Backend Engineer, my focus is not on the UI but on the data APIs that power these forms.

- **Input:** Authentication payloads (email/password), JSON documents representing clinical scores from the dashboard, and JSON objects representing quiz results from the Android app.
- **Output:** JWT tokens for session management, real-time JSON data streams for progress tracking, and HTTP status codes confirming database writes.

5. Implementation/Technological Environment

The implementation phase involved configuring the Firebase project and integrating the SDKs.

1. **Project Initialization:** A Firebase project was created via the Firebase Console.
2. **Firestore Setup:** The database was initialized in production mode. Composite indexes were built to support complex queries, such as fetching a patient's assessments ordered by date.
3. **Security Rules Implementation:** Rules were written to ensure that a parent can only read/write documents where the `parentId` matches their `auth.uid`, and therapists can only access records where `therapistId` matches their `auth.uid`.
4. **Hosting Configuration:** The Firebase CLI (`firebase-tools`) was installed. The web dashboard directory was initialized with `firebase init hosting`, and deployment scripts were created (`firebase deploy`).

6. Testing & Results

Backend testing focused on security, data integrity, and performance:

- **Authentication Testing:** Verified that users cannot access the database without a valid token. Tested role assignment to ensure a parent account cannot log into the therapist dashboard.
- **Security Rules Simulation:** Used the Firebase Emulator Suite to simulate unauthorized read/write attempts, confirming that the security rules successfully block malicious access.
- **Real-time Sync Testing:** Verified that when a child completes a quiz on the Android app, the resulting data is immediately reflected on the web dashboard without a page refresh.
- **Results:** The backend infrastructure proved highly resilient. Data synchronization occurred with sub-second latency, and all unauthorized access attempts were successfully rejected with `403 Permission Denied` errors.

7. Limitations

- **Offline Sync Conflicts:** While Firestore supports offline data access, complex conflict resolution (if multiple devices update the same document offline) requires careful handling on the client side.
- **Query Constraints:** Firestore's NoSQL nature means certain complex analytical queries (e.g., calculating the average CAP score across all patients in a specific age group) are inefficient and would require client-side processing or Cloud Functions.
- **Vendor Lock-in:** The architecture is tightly coupled with Google Firebase. Migrating to an alternative backend (like AWS or a custom Node.js server) would require significant rewriting of both the frontend and backend logic.

8. Conclusion & Future Scope

Conclusion: The implementation of a serverless Firebase backend successfully met all the requirements for the Sound for Silence platform. It provided a robust, secure, and real-time infrastructure that seamlessly bridged the Android application and the web dashboard. By leveraging Firebase Authentication, Firestore, and Hosting, the platform achieved high scalability and operational efficiency without the overhead of server management.

Future Scope:

- Integration of Firebase Cloud Functions to perform scheduled background tasks, such as sending email reminders to parents for upcoming therapy sessions.
- Implementing Firebase Storage to allow therapists to upload custom therapy resources or documents.
- Adding advanced analytics using Google Analytics for Firebase to track user engagement within the Android app.

BIBLIOGRAPHY

1. Google. (2025). *Firebase Documentation*. Retrieved from https://firebase.google.com/docs
2. Google. (2025). *Cloud Firestore Data Model*. Retrieved from https://firebase.google.com/docs/firestore/data-model
3. Google. (2025). *Firebase Security Rules*. Retrieved from https://firebase.google.com/docs/rules

APPENDIX A: CODING

A.1. Firebase Security Rules (firestore.rules)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Function to check if the user is authenticated
    function isAuth() {
      return request.auth != null;
    }

    match /users/{userId} {
      // Users can only read and write their own profile
      allow read, write: if isAuth() && request.auth.uid == userId;
    }

    match /patients/{patientId} {
      // Parents can read/write their child's data
      allow read, write: if isAuth() && resource.data.parentId ==
request.auth.uid;
      // Therapists can read/write data of patients assigned to them
      allow read, write: if isAuth() && resource.data.therapistId ==
request.auth.uid;

      match /assessments/{assessmentId} {
         // Sub-collection rules inherit the parent document's logic
conceptually,
         // but need explicit definition based on parent document
access.
         allow read, write: if isAuth(); // In production, use get() to
check parent owner
      }
    }

    match /stages/{stageId} {
      // Stages/Curriculum data is read-only for all authenticated
users
      allow read: if isAuth();
      allow write: if false; // Only updated via admin console
    }
  }
}
```

APPENDIX B: ABBREVIATIONS

API	Application Programming Interface
CAP	Categories of Auditory Performance
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity-Relationship
JSON	JavaScript Object Notation
JWT	JSON Web Token
NoSQL	Not Only SQL
RBAC	Role-Based Access Control
SIR	Speech Intelligibility Rating
UID	Unique Identifier